

Parella de Punts Més Propera i Més Llunyana

Pràctica 3

Serafí Nebot Ginard

Universitat de les Illes Balears (UIB)

Grau en Enginyeria Informàtica

Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java que resol el problema de la parella de punts més propera (*closest pair*) i més llunyana (*farthest pair*) sobre un conjunt de punts en el pla. S'implementen quatre algorismes: força bruta $O(n^2)$, divideix i venceràs amb franja ordenada $O(n \log^2 n)$, una variant amb *buckets*, i la parella més llunyana mitjançant *convex hull* i *rotating calipers* $O(n \log n)$. L'aplicació separa la lògica, la interfície gràfica i el control en tres capes independents, ofereix una interfície gràfica interactiva amb visualització dels punts i resultats, i incorpora una eina de benchmark amb gràfiques i predicció de temps. Es presenta una anàlisi experimental rigorosa que confirma les complexitats teòriques i revela que l'optimització per *buckets* no aporta guany pràctic per al problema de la parella més propera.

Termes d'índex—parella més propera, parella més llunyana, divideix i venceràs, força bruta, convex hull, rotating calipers, Model-Vista-Controlador, benchmark, Java Swing

1. Introducció

El problema de la parella de punts més propera (*closest pair problem*) és un problema fonamental de la geometria computacional: donat un conjunt de n punts en el pla, trobar la parella de punts amb la distància euclidiana mínima. La solució per força bruta examina totes les $\binom{n}{2}$ parelles amb cost $O(n^2)$, però l'algorisme de divideix i venceràs redueix el cost a quasi lineal; en aquesta implementació la franja s'ordena a cada nivell, donant $O(n \log^2 n)$ [1].

El problema complementari, la parella més llunyana (*farthest pair*), té la mateixa complexitat òptima $O(n \log n)$ però utilitza una estratègia diferent: primer es construeix el polígon convex que envolta tots els punts del núvol (*convex hull*) i després s'aplica la tècnica dels *rotating calipers* [2].

L'aplicació implementada permet generar núvols de punts amb quatre distribucions (uniforme, gaussiana, exponencial, agrupada), modificar paràmetres d'aquestes, executar els quatre algorismes, visualitzar els resultats gràficament i realitzar benchmarks sistemàtics amb predicció de temps.

2. Descripció de l'Aplicació

L'aplicació és una eina interactiva d'escriptori (Java Swing) que permet generar núvols de punts, trobar la parella més propera i més llunyana, i comparar el rendiment dels algorismes. La interfície es divideix en tres zones: un panell de controls a la part superior, la visualització de punts al centre i un panell de resultats a la dreta.

2.1. Generació de Punts

L'usuari configura el nombre de punts n amb un *spinner* i selecciona una de les quatre distribucions disponibles. Cada distribució té paràmetres configurables que es mostren dinàmicament:

- **Uniforme:** punts distribuïts uniformement dins del rang $[\min, \max]$ en ambdós eixos.
- **Gaussiana:** paràmetres de mitjana (fracció del rang), desviació estàndard σ (fracció del rang) i multiplicadors independents per a cada eix, permetent distribucions el·líptiques.
- **Exponencial:** factors λ_x i λ_y que controlen la densitat de punts cap a l'origen.
- **Agrupada (Clustered):** nombre de clusters (mínim 3), amb centres generats uniformement i punts distribuïts amb una gaussiana al voltant de cada centre.

2.2. Visualització del Núvol de Punts

El panell central mostra el núvol de punts generat. Cada punt es representa com un cercle sòlid de 4 px en gris fosc. El domini de coordenades $[0, 1000]^2$ es transforma linealment a les coordenades de pantalla amb un marge de 30 px a cada costat.

Quan s'executen els algorismes, les parelles resultants es ressalten sobre el núvol: la parella més propera en vermell i la més llunyana en blau. Cada ressaltat consisteix en una línia que connecta els dos punts i cercles de 8 px als extrems per destacar-los del núvol.

2.3. Execució i Resultats

El botó *Executar* llança els quatre algorismes en un fil de fons per no bloquejar la interfície. Durant l'execució, els controls es deshabiliten i el cursor canvia a mode d'espera. El botó *Aturar* permet cancel·lar l'execució cooperativament.

El panell de resultats a la dreta de la finestra mostra, per a cada algorisme, les coordenades i distància de la parella trobada i el temps d'execució mesurat. Mentre no s'ha executat cap algorisme, tots els valors mostren «—».

Cada algorisme s'executa amb el protocol de la Secció 3.2.4: escalfament JIT global al principi de la sèrie i, després, 1 execució mesurada per cas (algorisme, n).

2.4. Finestra de Benchmark

El botó *Benchmark* obre una finestra independent no modal per executar benchmarks sistemàtics. La finestra s'organitza en quatre zones:

- **Configuració (superior):** camps per introduir els valors de n separats per comes, un selector de distribució i els paràmetres de la distribució triada.
- **Gràfica (centre):** mostra les quatre corbes de temps en funció de n . Permet activar escala logarítmica i amagar series individuals. Superposa corbes ajustades (línia discontinua) un cop finalitzat el benchmark.
- **Predicció (dreta):** un cop finalitzat el benchmark, ajusta les constants multiplicatives dels models teòrics i permet introduir un valor de n arbitrari per predir el temps d'execució de cada algorisme.
- **Taula de dades (inferior):** mostra els temps mesurats per a cada n i algorisme en format tabular.

3. Disseny i Implementació

3.1. Estructura de l'Aplicació

L'aplicació es divideix en tres capes amb responsabilitats ben separades, que es comuniquen per events:

- 1) **Model:** la classe `Model` encapsula el núvol de punts, els resultats de cada algorisme (parella i temps), i els resultats del benchmark. Només emmagatzema dades; els algorismes viuen en classes separades dins la mateixa capa.
- 2) **Vista:** conjunt de components Swing que presenten els controls, la visualització i la gràfica. No contenen lògica de negoci ni dades del problema.
- 3) **Controlador:** implementa la interfície `ViewListener` per rebre els events de la vista, gestiona el cicle de vida dels fils de fons i actualitza el model i la vista.

La interfície `ViewListener` defineix quatre events: `onGenerate`, `onRun`, `onStop` i `onOpenBenchmark`. La vista emet events sense conèixer qui els processa, i el controlador implementa la interfície sense conèixer els components gràfics interns de la vista.

Degut a que la vista actualitza automàticament el núvol de punts cada vegada que l'usuari modifica qualche dada de configuració, existeix un temporitzador *debounce* de 300 ms que evita emetre l'event `onGenerate` a cada tecleig; la vista només el llança quan l'usuari ha deixat d'interactuar durant 300 ms.

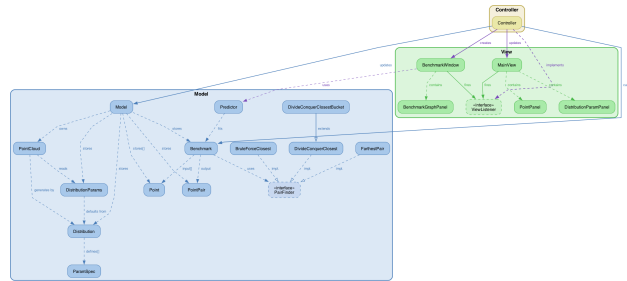


Figura 1. Diagrama de classes de l'aplicació amb les relacions entre les tres capes.

3.2. Capa del Model

La capa del model conté les classes de dades i els algorismes. No importa cap classe de la vista ni del controlador.

3.2.1. Model. La classe `Model` és el contenidor central d'estat de l'aplicació. Emmagatzema el núvol de punts actual (`Point[]`), la distribució i els paràmetres usats per generar-lo (`Distribution`, `ParamSpec/DistributionParams`), els resultats de cada algorisme (`PointPair` i temps en ms) i els resultats del darrer benchmark (`BenchmarkEntry[]`). No conté cap lògica d'algorisme: és un magatzem de dades pur que el controlador escriu i la vista llegeix a través dels mètodes d'accés.

3.2.2. Point i PointPair. `Point` és un registre immutable amb dos camps `double` (`x`, `y`). Ofereix `distanceTo()` que calcula la distància euclidiana entre dos punts.

`PointPair` encapsula una parella de punts i calcula la distància de forma *lazy*: el camp `distance` s'inicialitza a `-1` com a sentinella i es calcula només la primera vegada que es demana. Gràcies a això, els algorismes que construeixen molts `PointPair` intermedis només paguen el cost de l'arrel quadrada quan realment es necessita la distància.

3.2.3. Distribution, PointCloud, DistributionParams i ParamSpec. L'enumeració `Distribution` defineix les distribucions disponibles (uniforme, gaussiana, exponencial i agrupada) i el seu nom de presentació. `PointCloud` implementa aquestes distribucions usant `java.util.Random`.

Els paràmetres de cada distribució es defineixen com a arrays de `ParamSpec` (registre amb nom i valor per defecte), i els valors actuals s'encapsulen en `DistributionParams` (registre immutable amb còpia defensiva al constructor). Aquest disseny permet que la vista construeixi els camps

de configuració automàticament a partir de `ParamSpec[]`, sense conèixer quines distribucions existeixen ni quants paràmetres té cadascuna. D'aquesta forma, si en el futur s'afageix, elimina o es modifica qualche distribució o paràmetre no fa falta modificar la vista, aquesta mostrarà totes les opcions automàticament.

3.2.4. Benchmark i Predictor. La classe `Benchmark` centralitza el protocol de mesura i l'execució paral·lela. El protocol és:

- 1) **Escalfament JIT global:** s'executen tots els algorismes 10 vegades amb $n = 500$ abans d'iniciar una sèrie de benchmark (finestra/CLI) per forçar la compilació JIT i així descartar possibles interferències amb el temps d'execució mesurat.
- 2) **Mesura:** una execució mesurada amb `System.nanoTime()`, convertida a mil·lisegons.

Per a l'execució individual des de la interfície principal s'aplica directament la fase de mesura.

Per al benchmark en sèrie, `Benchmark.runSeries` genera els arrays de punts una sola vegada al fil principal i després distribueix les tasques amb un `ExecutorService` de granularitat fina: una tasca per parella (algorisme, n), de manera que els quatre algorismes per al n més gran s'executen en paral·lel en nuclis separats.

La classe `Predictor` ajusta una constant multiplicativa a per a cada algorisme amb mínims quadrats ponderats (sense ordenada a l'origen):

$$a = \frac{\sum_i t_i \cdot f(n_i)}{\sum_i f(n_i)^2} \quad (1)$$

on n_i i t_i són la quantitat de punts del núvol i el temps d'execució mesurat per a l'execució i del benchmark. La funció $f(n_i)$ equival al temps teòric per a cada un dels algorismes. La ponderació dóna més pes a les mesures amb n gran, on el comportament asimptòtic domina la sobrecàrrega de la JVM.

3.2.5. Interfície Comuna dels Algorismes. Tots els algorismes implementen la mateixa interfície `PairFinder`:

```
1 public interface PairFinder {
2     PointPair find(Point[] points);
3 }
```

Listing 1. Interfície comuna per a tots els algorismes.

Això permet al benchmark i al controlador cridar qual-sevol algorisme sense saber quin és concretament.

Pel que fa a les dues variants de D&C, `DivideConquerClosest` implementa l'algorisme complet (ordenar per X , recursió, construir franja, processar franja). `DivideConquerClosestBucket` en reutilitza tot menys l'últim pas: sobreescrui només el mètode `processStrip()` per usar `buckets` en lloc d'ordenar per Y . D'aquesta manera no es repeteix cap codi: tot el que és comú entre les dues variants viu en un sol lloc.

3.2.6. Algorisme de Força Bruta. L'algorisme de força bruta (`BruteForceClosest`) examina totes les $\binom{n}{2} = \frac{n(n-1)}{2}$ parelles de punts i retorna la de distància mínima.

```
1 for (int i = 0; i < n - 1; i++) {
2     for (int j = i + 1; j < n; j++) {
3         double dist = points[i].distanceTo(points[j]);
4         if (dist < bestDist) {
5             bestDist = dist;
6             bestP1 = points[i];
7             bestP2 = points[j];
8         }
9     }
10 }
```

Listing 2. Nucli de l'algorisme de força bruta.

Cost temporal. L'algorisme avalua totes les $\binom{n}{2} = \frac{n(n-1)}{2}$ parelles, per tant $T(n) \in O(n^2)$.

Cost espacial. L'espai addicional és constant, ja que només es mantenen variables auxiliars i la millor parella: $O(1)$.

Notes pràctiques. La implementació comprova `Thread.interrupted()` a cada iteració del bucle extern per permetre la cancel·lació cooperativa des de la interfície. La mateixa estratègia (amb punts de comprovació adaptats a cada bucle/recursió) s'aplica també a D&C, D&C Bucket i parella llunyana.

3.2.7. Algorisme de Divideix i Venceràs. L'algorisme de divideix i venceràs (`DivideConquerClosest`) segueix les següents passes:

- 1) Ordenar els punts per coordenada x una sola vegada ($O(n \log n)$).
- 2) Recursivament trobar la parella més propera a la meitat esquerra i la meitat dreta.
- 3) Construir la franja central de punts amb $|x - x_{\text{mid}}| < d_{\text{min}}$.
- 4) Processar la franja: ordenar per y i per a cada punt, comparar amb els 7 veïns següents (argument de l'empaquetament).

El cas base s'activa quan el subproblema té 3 o menys punts, resolent-se per força bruta amb cost $O(1)$.

Argument d'empaquetament (per què 7 veïns). Després de resoldre les dues meitats, d_{min} és la millor distància dins de cada costat. En la franja central (amplada $2d_{\text{min}}$), per a un punt fixat només poden ser candidats els punts amb diferència en y menor que d_{min} . Això defineix un rectangle de mida $2d_{\text{min}} \times d_{\text{min}}$. Si el subdividim en cel·les de costat $d_{\text{min}}/2$, obtenim 8 cel·les; no hi pot haver dos punts a la mateixa cel·la perquè estarien a distància menor que d_{min} (contradicció). Per tant, hi ha com a màxim 8 punts rellevants en aquesta zona i, ordenant per y , n'hi ha prou de comparar cada punt amb els 7 següents. Aquest límit constant (6–8 segons la formulació geomètrica) és la clau del cost lineal de la franja [1].

Cost temporal. Podem expressar el cost de la funció recursiva de la següent forma:

$$T(n) = 2T(n/2) + f(n) \quad (2)$$

on $f(n) \in O(n \log n)$ prové del processament de la franja a cada nivell: ordenació per y ($O(k \log k)$ amb $k \leq n$) i comparacions acotades entre veïns a la franja (cost lineal sobre k , amb un màxim constant de comparacions per punt).

Podem resoldre la recurrència amb el *Teorema Mestre*:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

on a és el nombre de subproblemes, n/b la mida de cada subproblema, i $f(n)$ el cost de combinar resultats.

En el nostre cas:

- $a = 2$ (dues meitats),
- $b = 2$ (cada meitat té mida $n/2$),
- $f(n) = n \log n$ (cost de processar la franja).

Com que el treball recursiu pur és d'ordre n (ja que $n^{\log_2 2} = n^1 = n$) i aquí la combinació és un factor $\log n$ més gran ($n \log n$), la forma generalitzada del Teorema Mestre dona:

$$T(n) \in O(n \log^2 n)$$

Cost espacial. L'espai és $O(n)$ (buffer de franja preallocat + pila recursiva).

Notes pràctiques. Les optimitzacions següents no canvien l'ordre asimptòtic, però redueixen constants i milloren el rendiment real de l'algorisme.

Optimització 1 – Cerca binària per als límits de la franja.

En lloc d'un escaneig lineal $O(k)$ per trobar els punts dins de la franja, s'utilitza cerca binària $O(\log k)$ amb els mètodes `lowerBound` i `upperBound`, aprofitant que els punts ja estan ordenats per x . Això no millora la complexitat asimptòtica (el pas dominant segueix sent l'ordenació de la franja) però redueix el treball constant.

Optimització 2 – Buffer preallocat per a la franja.

A cada nivell de recursió, l'algorisme necessita copiar els punts de la franja central a un buffer temporal per ordenar-los per y . La implementació ingènua crearia un nou array a cada crida recursiva, generant $O(\log n)$ al·locacions (una per nivell).

Per eliminar la pressió sobre el *garbage collector* i millorar la localitat de cache, s'al·loca un únic array `Point[n]` al mètode `find()` i es reutilitza a tots els nivells de recursió. El buffer és segur perquè cada nivell de recursió l'omple amb `System.arraycopy` i només accedeix als índexos $[0, k)$ on k és el nombre de punts de la franja actual. Els nivells de recursió no es trepitgen perquè l'execució és seqüencial (un sol fil): un nivell pare copia la seva franja, fa la crida al nivell fill i queda aturat fins que el fill retorna. Si el fill sobreescriu el buffer, no passa res: el pare ja no necessita les dades antigues.

3.2.8. Variant amb Buckets. La variant amb *buckets* (`DivideConquerClosestBucket`) intenta millorar el pas de processament de la franja. En lloc d'ordenar la franja per y amb cost $O(k \log k)$, distribueix els punts en cubetes (*buckets*) d'alçada $d_{\min}$ i comprova només la cubeta actual i la immediatament superior. Per l'argument de l'empaquetament explicat abans, cada cubeta conté com

a màxim 8 punts, de manera que el cost per punt és $O(1)$ i el cost total de la franja és $O(k)$.

Cost temporal. La millora damunt del D&C fa que la recurrència es redueixi de $T(n) = 2T(n/2) + k \log k$ a $T(n) = 2T(n/2) + k$. Com que $k \leq n$, llavors $T(n) \in O(n \log n)$.

Cost espacial. El consum addicional és lineal en el pitjor cas ($O(n)$ per al buffer i estructures de franja), però amb constants més altes que D&C per la gestió de cubetes.

Notes pràctiques. Cost de Cubetes Disperses. L'optimització per *buckets* requereix crear un array de cubetes indexat pel rang de coordenada y . El nombre de cubetes és:

$$\text{numBuckets} = \left\lceil \frac{y_{\max}}{d_{\min}} \right\rceil - \left\lceil \frac{y_{\min}}{d_{\min}} \right\rceil + 1 \quad (3)$$

Quan `numBuckets` $\gg k$ (molt més cubetes que punts), la representació en memòria esdevé molt dispersa: es creen vectors de grandària proporcional a `numBuckets` (p. ex. `buckets[numBuckets * 8]` i `counts[numBuckets]`), però només s'omplen k posicions útils. Això implica dos efectes pràctics: (1) sobrecost d'al·locació/inicialització d'estructures grans i (2) pitjor localitat espacial, perquè es recorren moltes cubetes buides entre poques cubetes amb dades. En conseqüència augmenten les fallades de cache i la pressió de memòria, i el guany teòric del pas lineal queda neutralitzat o superat. Aquest patró també apareix en índexs espacials basats en graella i *spatial hashing*.

Per a evitar aquest problema, es podria afegir una guarda (p. ex. `if (numBuckets > k)`) per fer *fallback* al processament clàssic de franja. Aquesta decisió reduiria el sobrecost de memòria, però també faria que la variant *bucket* es comporti com el D&C normal en la majoria de casos pràctics. A n'aquesta implementació no s'ha introduït cap guarda per a mantenir el comportament de la variant de *buckets*.

3.2.9. Parella Més Llunyana. L'algorisme de la parella més llunyana (`FarthestPair`) combina dues tècniques: *convex hull* i *rotating calipers* [2].

Fase 1 – Convex hull (envolupant convexa). Primer s'ordenen els punts per coordenada x i es construeix la frontera exterior del núvol: polígon convex que conté la resta de punts al seu interior. El codi ho fa en dues passades (cadena inferior i cadena superior), eliminant els punts que fan gir no convex. El resultat és una llista ordenada de vèrtexs de la frontera; tots els punts interiors queden descartats perquè no poden ser extrems de la distància màxima. L'ordenació inicial domina el cost amb $O(n \log n)$; la construcció de les dues cadenes és lineal. Per tant, el cost de la fase és $O(n \log n)$.

Fase 2 – Rotating calipers. Sobre la frontera convexa, es mantenen dos índexs (dos "calipers") i es recorren els vèrtexs buscant parelles antipodals. A cada pas es compara l'àrea orientada del costat actual amb el següent per decidir quin punter avançar. Això evita provar totes les parelles de vèrtexs: cada punter fa una volta com a màxim, de manera que la cerca sobre l'envolupant és lineal. Si h és el nombre

de vèrtexs de l'envolupant convexa, cada punter avança com a màxim h posicions i el cost és $O(h)$ (amb $h \leq n$).

En resum funcional, el procés és:

- 1) Reduir el problema als vèrtexs de la frontera (*convex hull*).
- 2) Cercar la millor parella només entre aquests vèrtexs (*rotating calipers*).

Cost temporal. Com que el *convex hull* costa $O(n \log n)$ i els *rotating calipers* costen $O(h)$, el cost total és $T(n) \in O(n \log n)$.

Cost espacial. L'espai addicional és lineal per emmagatzemar l'envolupant convexa i estructures auxiliars: $O(n)$. La raó és que, en el pitjor cas, tots els punts poden formar part de l'envolupant (per exemple, si ja estan sobre una frontera convexa), de manera que cal guardar fins a n vèrtexs. A més, les estructures temporals de construcció (cadena inferior/superior) també contenen com a màxim una quantitat proporcional a n punts.

3.3. Capa de la Vista

La capa de la vista consta de cinc components Swing principals, cadascun amb una responsabilitat concreta.

3.3.1. MainView. La finestra principal usa BorderLayout: el panell de controls a dalt (NORTH), el PointPanel al centre i el panell de resultats a la dreta (EAST). El panell de controls combina dos subpanells amb FlowLayout: un a l'esquerra amb el JSpinner de n , el JComboBox de distribució i el DistributionParamPanel, i un a la dreta amb els botons *Executar*, *Aturar* i *Benchmark*.

La comunicació cap al controlador es fa exclusivament a través de ViewListener: el botó *Executar* crida `onRun`, *Aturar* crida `onStop`, i *Benchmark* crida `onOpenBenchmark`. La vista no coneix el controlador; simplement invoca l'event sobre el listener registrat.

El panell de resultats organitza vuit etiquetes en vertical amb BoxLayout: quatre per a les parelles trobades (coordenades i distància) i quatre per als temps d'execució. Quan s'ha generat un núvol nou però encara no s'han executat algorismes, les etiquetes mostren «—» i el botó *Executar* s'habilita per indicar que les dades estan llestes.

3.3.2. PointPanel. El panell on es mostra el núvol de punts generat i la solució trobada per a la distància mínima i màxima. El renderitzat es fa directament a paintComponent amb tres capes seqüencials: primer tots els punts del núvol en gris, després la parella llunyana en blau (si disponible) i finalment la parella propera en vermell (si disponible).

La transformació de domini a pantalla és:

$$x_{px} = \text{MARGIN} + \frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot w \quad (4)$$

$$y_{px} = \text{MARGIN} + h - \frac{y - y_{\min}}{y_{\max} - y_{\min}} \cdot h \quad (5)$$

on w i h són les dimensions útils del panell (descomptant els marges). Els valors $x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$ representen els límits del domini de dades (mínim i màxim de cada eix) utilitzats per normalitzar les coordenades abans de pintar-les. La resta de h a la fórmula de y inverteix l'eix perquè la pantalla té l'origen a dalt i el domini matemàtic a baix.

3.3.3. DistributionParamPanel. Panell auxiliar encarregat de mostrar i editar els paràmetres de la distribució seleccionada.

Aquest panell genera els seus camps de text dinàmicament en el moment en què es canvia la distribució seleccionada. Recorre l'array `ParamSpec[]` de la distribució i crea un parell JLabel + JTextField per a cada paràmetre, inicialitzant-lo amb el valor per defecte de l'especificació. Cada camp té un DocumentListener que, quan el contingut canvia, reinicia el temporitzador de *debounce* de MainView, el qual al seu torn emet l'event `onGenerate` al ViewListener un cop transcorreguts els 300 ms. Si l'usuari introdueix un valor no numèric, el mètode `read()` retorna silenciosament el valor per defecte sense mostrar cap error.

Gràcies a aquest disseny, afegir una nova distribució o modificar els seus paràmetres no requereix cap canvi a la vista.

3.3.4. BenchmarkWindow. Finestra secundària on es configura i s'executa el benchmark, i on es visualitzen els resultats agregats.

La finestra de benchmark és un JDialog no modal. Utilitza BorderLayout amb la configuració a dalt, la gràfica al centre, un panell d'opcions i predicció a la dreta, i la taula de dades a baix.

De manera idèntica a MainView, BenchmarkWindow es comunica amb el controlador a través de ViewListener. El mètode `onRunBenchmark()` forma part de la mateixa interfície que `onRun`, `onStop`, etc., de manera que el controlador és l'únic punt d'implementació i la finestra no coneix en cap moment el seu tipus concret.

L'execució del benchmark es delega a un SwingWorker. Quan el SwingWorker acaba, el mètode `done()` s'executa a l'EDT, actualitza la gràfica, la taula i ajusta les constants del predictor, i habilita el botó de predicció.

3.3.5. BenchmarkGraphPanel. Component de dibuix encarregat de representar les corbes de temps mesurat i les corbes ajustades pel predictor.

La gràfica es dibuixa completament a paintComponent, sense cap llibreria externa. El mètode `generateRoundTicks()` calcula etiquetes d'eixos arrodonides a potències de $10 \times \{1, 2, 5\}$ per obtenir gralles netes independentment del rang de valors.

En mode logarítmic, l'eix y es transforma amb $\log_{10}$ i les divisions secundàries es col·loquen als múltiples $\times 2$ i $\times 5$ de cada potència de 10. En mode lineal, les divisions es calculen amb `generateRoundTicks`.

Les corbes ajustades es dibuixen amb una línia discontinua (BasicStroke amb array de *dash*) en un to lleugerament més fosc que la sèrie de dades corresponent, mostrant el model $a \cdot f(n)$ mostrejat a un píxel per columna.

3.4. Capa del Controlador

La capa del controlador consisteix en una sola classe, Controller, que implementa ViewListener i conté tota la lògica d'orquestració de l'aplicació.

3.4.1. Gestió dels Events de la Vista. Cada mètode de ViewListener correspon a una acció de l'usuari:

- **onGenerate:** crida `model.generatePoints()` amb els paràmetres rebuts, i seguidament `view.showPoints()` per actualitzar la visualització i `view.clearResults()` per esborrar els resultats anteriors. Finalment habilita el botó *Executar*.
- **onRun:** crea un `SwingWorker` que executa els quatre algorismes sobre el núvol actual al fil de fons, desa els resultats al model amb `model.setResults()`, i en acabar crida `view.updateResults()` a l'EDT. Durant l'execució, crida `view.setRunning(true)` per deshabilitar els controls i activar el cursor d'espera.
- **onStop:** crida `worker.cancel(true)`, que interromp el fil de fons. La cancel·lació és cooperativa: tots els algorismes comproven periòdicament `Thread.isInterrupted()` en punts de control (bucles principals i entrades recursives) i llancen una excepció quan detecten la interrupció.
- **onOpenBenchmark:** crea la `BenchmarkWindow` (si no existeix ja) passant-se a si mateix com a `ViewListener`, i la fa visible.

3.5. CLI de Benchmark

A més de la interfície gràfica, s'ofereix una eina de línia de comandes (`BenchmarkCLI`) que permet executar benchmarks amb paràmetres configurables i sortida en format taula o CSV:

El flux d'execució del CLI és el següent:

- 1) Llegeix els paràmetres de consola (distribució, valors de n , algorismes, nombre de repeticions i format de sortida).
- 2) Mostra la configuració seleccionada i fa un escalfament JIT inicial amb un conjunt petit de punts.
- 3) Per a cada repetició, genera els núvols de punts per a tots els valors de n i llança en paral·lel les tasques (algorisme, n).
- 4) Recull els temps, en calcula la mitjana entre repeticions i imprimeix el resultat final en taula o CSV.

Paràmetres principals:

- `-d/-dist`: distribució (UNIFORM, GAUSSIAN, EXPONENTIAL, CLUSTERED).
- `-p/-params`: paràmetres numèrics de la distribució (en l'ordre definit per cada distribució).
- `-n/-nvals`: llista de mides de problema separades per comes.
- `-a/-algos`: subconjunt d'algorismes (brute, dc, bucket, farthest).
- `-r/-runs`: nombre d'execucions independents per fer mitjana.
- `-csv`: sortida en format CSV (si no, taula textual).

Per evitar ambigüitats, la Taula 1 resumeix l'ordre exacte dels valors a `-p/-params` per a cada distribució.

Taula 1. ORDRE DE PARÀMETRES PER A `-p/-PARAMS`.

Distribució	Ordre de paràmetres a <code>-p</code>
UNIFORM	min,max
GAUSSIAN	mean,sigma,mulX,mulY
EXPONENTIAL	lambdaX,lambdaY
CLUSTERED	clusters,spread

```

1 # Tots els algorismes, distribucio uniforme
2 java -cp target/classes \
3   com.serafinebot.p3.BenchmarkCLI
4
5 # D&C vs Bucket, gaussiana, CSV
6 java -cp target/classes \
7   com.serafinebot.p3.BenchmarkCLI \
8   -a dc,bucket -d GAUSSIAN --csv
9
10 # 3 execucions promitjades
11 java -cp target/classes \
12   com.serafinebot.p3.BenchmarkCLI -r 3

```

Listing 3. Exemples d'ús del CLI.

4. Anàlisi Experimental

Totes les mesures s'han realitzat amb el CLI de benchmark, amb 3 execucions independents mitjanes. S'han provat les distribucions uniforme, gaussiana ($\mu = 0.5$, $\sigma = 1/6$), exponencial ($\lambda_x = \lambda_y = 5$) i agrupada (5 clusters). L'execució del benchmark distribueix les tasques entre nuclis; això pot afegir soroll en temps molt petits però no afecta les tendències globals. Com a cas addicional s'ha provat una gaussiana estirada ($\sigma = 0.05$, multiplicadors $\times 4$ i $\times 0.25$). Els valors analitzats cobreixen des de $n = 500$ fins a $n = 300\,000$, amb més densitat de mostres a valors petits i intermedis. Els temps petits (< 0.1 ms) presenten variabilitat elevada; s'analitzen sobretot tendències i valors per a n grans. Cal remarcar que la generació de núvols és aleatòria i, en aquesta versió, no es fixa una llavor (*seed*) global; per tant, dues execucions amb els mateixos paràmetres produeixen mostres diferents. Per això es realitzen 3 execucions seguides per a cada configuració de cada algorisme i es fa feina damunt la seva mitja.

4.1. Distribució Uniforme: Comparació Completa

La Taula 2 recull els temps mitjans per a la distribució uniforme i la Figura 2 mostra el creixement en escala logarítmica.

Taula 2. TEMPS D'EXECUCIÓ (MS) PER A DISTRIBUCIÓ UNIFORME.

n	Bruta	D&C	D&C Bkt.	Llunyana
500	0.178	0.472	1.324	0.848
1000	0.685	0.889	1.137	2.331
5000	27.263	7.542	51.043	50.777
10000	171.028	18.215	28.158	28.750
50000	9180.847	90.739	274.017	135.219
100000	36 675.313	345.969	493.501	394.892
150000	87 350.465	596.639	1008.263	491.157
200000	143 702.082	878.369	1692.344	868.165
250000	175 852.252	1260.477	2369.061	987.417
300000	209 233.170	1701.369	2783.924	1046.394

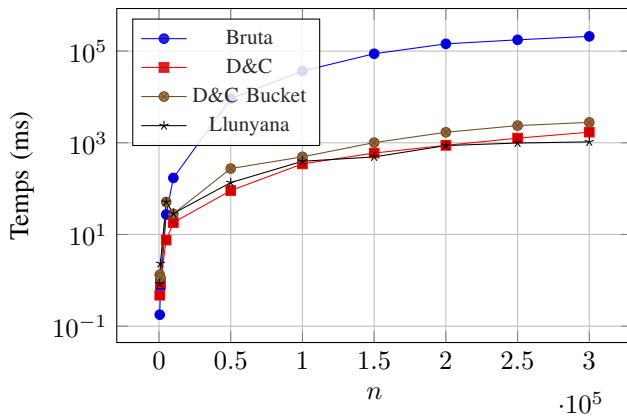


Figura 2. Distribució uniforme: temps vs n (escala logarítmica).

Els resultats mostren el patró esperat: força bruta domina per a n petits per la seva baixa constant, però queda ràpidament superada per D&C i la parella llunyana. Per $n = 300\,000$, D&C és $\approx 123\times$ més ràpid que força bruta, mentre que la parella llunyana és la més ràpida dels algorismes quasi-lineals. La variant amb *buckets* és sistemàticament més lenta que D&C en la distribució uniforme ($\sim 64\%$ a $n = 300\,000$).

4.2. Comparació entre Distribucions

La Taula 3 resumeix els temps a $n = 300\,000$ per a totes les distribucions i el cas gaussià estirat. La Figura 3 mostra la comparativa completa dels quatre algorismes.

Taula 3. TEMPS (MS) PER ALGORISME I DISTRIBUCIÓ A $n = 300\,000$.

Distribució	Bruta	D&C	D&C Bkt.	Llunyana
Uniforme	209 233.2	1701.4	2783.9	1046.4
Gaussiana	215 653.4	1662.1	3101.5	1195.7
Exponencial	210 040.3	2021.5	4125.5	1338.5
Agrupada (5)	209 850.6	1721.2	3322.0	1102.0
Gaussiana est.	212 501.2	1372.6	1702.7	925.6

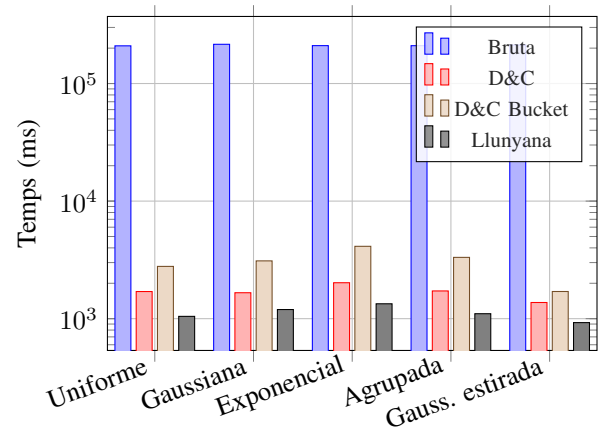


Figura 3. Comparativa per distribució i algorisme a $n = 300\,000$ (escala logarítmica).

Amb aquesta comparativa completa es veu més clarament que la distribució exponencial és la més costosa en els tres algorismes quasi-lineals (D&C, D&C Bucket i Llunyana). En força bruta, en canvi, la variació entre distribucions és pràcticament nul·la perquè comprova tots els punts de la mateixa forma independentment de la distribució.

La distribució gaussiana estirada (ell·líptica) serveix per comprovar si aquesta geometria altera el comportament. En aquest cas el rang efectiu en l'eix y és més petit i el nombre de cubetes baixa, de manera que la variant amb *buckets* s'apropa més a D&C que en la resta de distribucions. Tot i això, per al rang analitzat continua sent més lenta (1702.71 ms vs 1372.55 ms a $n = 300\,000$). El comportament no és monòton per a tots els n , mostrant sensibilitat a la geometria i al soroll de mesura.

4.3. Problemes Pràctics de la Variant amb Buckets

La Figura 4 mostra el ràtio Bucket/D&C per a la distribució uniforme i la gaussiana estirada. En uniforme el ràtio és major que 1 per a n grans, evidenciant el sobrecost d'al·locar cubetes disperses. En la gaussiana estirada el ràtio pot baixar per sota d'1 quan el rang efectiu en y és reduït, però el patró és irregular. Els valors erràtics per a n petits reflecteixen l'overhead de construir les cubetes quan el temps absolut és molt baix i el soroll de mesura domina.

En proves addicionals sobre gaussiana no estirada amb mitjana=0.5, desv.=0.15, mulX=10 i mulY=0.5 (paràmetres passats com `-p 0.5,0.15,10,0.5`) i mides fins a $n = 1\,000\,000$ (comparant només dc i bucket), s'observa que en alguns casos puntuals la variant bucket pot tenir un rendiment lleugerament per damunt de D&C. Això és compatible amb la naturalesa aleatòria del mostreig: petites variacions en la disposició dels punts poden alterar la geometria de franja i el cost efectiu de cubetes.

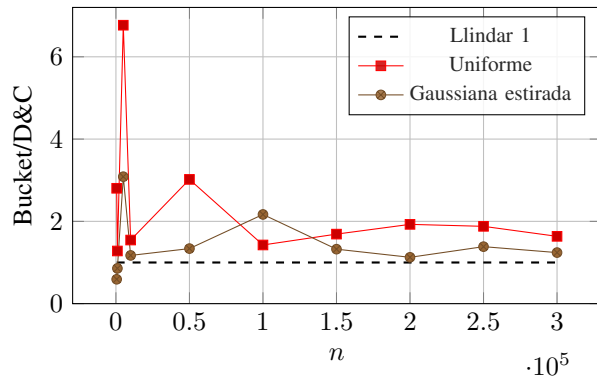


Figura 4. Ràtio Bucket/D&C per n .

5. Conclusió i Opinió Personal

En aquesta pràctica s'ha implementat una aplicació completa per al problema de la parella de punts més propera i més llunyana, amb una arquitectura MVC clara, visualització gràfica i una eina de benchmark específica. El disseny ha permès separar la generació de punts, l'execució dels algorismes i la presentació de resultats sense acoblaments innecessaris.

Els resultats experimentals confirmen la transició esperada entre força bruta i els algorismes quasi-lineals: la força bruta és competitiva per a n petits per la seva baixa constant, però D&C domina a partir de valors moderats i la parella llunyana és especialment eficient per a n grans. La variant amb *buckets* mostra un sobrecost important en distribucions contínues, degut a cubetes disperses i pressió sobre el GC; només en configuracions específiques arriba a apropar-se a D&C.

Des d'un punt de vista personal, la part més útil del projecte ha estat disposar d'un CLI de benchmark reproduïble amb sortida CSV. Aquesta eina ha reduït molt el temps d'anàlisi respecte a mesurar des de la GUI, perquè ha permès repetir sèries completes de proves, comparar distribucions amb la mateixa configuració i actualitzar les figures de la memòria de forma ràpida. També ha facilitat detectar que la variant bucket no falla per teoria, sinó per costos pràctics de memòria i localitat.

Durant el desenvolupament del programa es va iniciar una implementació de la variant D&C amb *KD-tree*. Però, en dedicar una part important del temps a l'anàlisi i a la millora de la variant amb *buckets*, finalment es va optar per descartar-la. Com a treball futur, seria interessant reprendre aquesta variant, avaluar-la amb el mateix protocol experimental i identificar en quins escenaris pot oferir un millor rendiment.

Referències

- [1] GeeksforGeeks, "Closest Pair of Points using Divide and Conquer algorithm," 2025. [Online]. Available: <https://www.geeksforgeeks.org/closest-pair-of-points-onlogn-implementation/>
- [2] GeeksforGeeks, "Convex Hull using Graham Scan," 2025. [Online]. Available: <https://www.geeksforgeeks.org/convex-hull-using-graham-scan/>